

**GamutX**

**List**

Python Basics

# Introduction to List

- What is a List?
- How to Create a List?
- Types of List

# What is a List?

- A List is a **Collection of Values Stored in a Single Variable**
- Items can be of any data type — **numbers, strings, or even other lists**



## Grocery List

- Apple
- Broccoli
- Eggs
- Cheese
- Bread



## Travel Packing List

- Shirt
- Pants
- Socks
- Toothbrush
- Phone charger

# How to Create a List

To create a list, we write items inside **square brackets []**, separated by commas.

## Syntax

```
list_name = [item1, item2, item3]
```

the elements in the list

## Creating a List with initial value

```
fruits = ["apple", "banana", "cherry"]  
print(fruits)
```

# Output: ['apple', 'banana', 'cherry']



# Types of List

- There are three types of List

Empty List

Homogeneous List

Heterogeneous List

## Empty List

- An empty **list doesn't contain any items.**
- We can create it first and add items later.

```
empty_list = [] ←--- Empty List  
print(empty_list) # Output: []
```

## Homogeneous List

- A homogeneous list **contains elements that are all of the same data type.**

### List of Integers

```
int_list = [1, 2, 3, 4, 5]
print(int_list)
# Output: [1, 2, 3, 4, 5]
```

### List of Strings

```
str_list = ["apple", "banana", "cherry"]
print(str_list)
# Output: ['apple', 'banana', 'cherry']
```

### List of Floats

```
float_list = [1.1, 2.2, 3.3, 4.4]
print(float_list)
# Output: [1.1, 2.2, 3.3, 4.4]
```

### List of Booleans

```
bool_list = [True, False, True]
print(bool_list)
# Output: [True, False, True]
```

## Heterogeneous List

- A heterogeneous list **contains elements of different data types**.

### Mixed List with Integers, Strings, and Floats

```
mixed_list1 = [1, "hello", 3.14, 42]  
print(mixed_list1)
```

```
# Output: [1, 'hello', 3.14, 42]
```

### Mixed List Including Booleans

```
mixed_list2 = [10, "Python", 3.1415, True, False]  
print(mixed_list2)
```

```
# Output: [10, 'Python', 3.1415, True, False]
```

## Summary

- **Empty List** → Contains no elements
- **Homogeneous List** → All elements are of the same data type
- **Heterogeneous List** → Elements are of mixed data types

## List Indexing

- What is List Indexing?
- Positive Indexing
- Negative Indexing

# What is List Indexing?

- List indexing allows us to **access individual elements in a list** using their position.
- Python uses **zero-based indexing**, which means **counting starts at 0**.

## Syntax

```
list_name[index]
```

The position of the item we want to access

## Example

```
animals = ["lion", "tiger", "bear", "wolf"]  
  
print(animals[0]) # Output: lion  
print(animals[2]) # Output: bear
```

First Element

Last Element

[ "lion", "tiger", "bear", "wolf" ]

0

1

2

3

Indexing



# Concept Practice Task

## Indexing with Numbers

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0]) # Output: 10 (First element)
print(numbers[2]) # Output: 30 (Third element)
print(numbers[4]) # Output: 50 (Fifth element)
```

## Accessing the Last Element

```
cities = ["New York", "London", "Tokyo", "Sydney"]

print(cities[3])
# Output: Sydney (Fourth element)
```

## Indexing with Mixed Data Types

```
mixed_list = [1, "hello", 3.14, True]

print(mixed_list[0]) # Output: 1 (First element)
print(mixed_list[1]) # Output: hello (Second element)
print(mixed_list[2]) # Output: 3.14 (Third element)
print(mixed_list[3]) # Output: True (Fourth element)
```

## Positive Indexing

- Starts **from 0 (left to right)**
- Used to **access elements from the beginning** of the list

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
print(fruits[0]) # Output: apple
```

```
print(fruits[2]) # Output: cherry
```

["apple", "banana", "cherry", "date"]

0

1

2

3

Positive

Left to right →

## Negative Indexing

- Starts **from -1 (right to left)**
- Used to **access elements from the end** of the list

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
```

```
print(fruits[-1]) # Output: elderberry
```

```
print(fruits[-3]) # Output: cherry
```

["apple", "banana", "cherry", "date", "elderberry"]

-5

-4

-3

-2

-1

Negative

Right to Left ←

# Concept Practice Task - Positive

## Accessing Elements in a List of Numbers

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1]) # Output: 20
print(numbers[4]) # Output: 50
```

## Accessing Elements in a List of Mixed Types

```
varied_list = [1, "string", 3.14159, False, None]

print(varied_list[2]) # Output: 3.14159
print(varied_list[0]) # Output: 1
```

## Accessing Elements in a List of Decimal Numbers

```
decimals = [0.1, 1.5, 3.6, 7.8]

print(decimals[1]) # Output: 1.5
print(decimals[3]) # Output: 7.8
```

## List Length with Positive Indexing

```
planets = ["Mercury", "Venus", "Earth", "Mars"]

print(planets[len(planets) - 1])

# Output: Mars
```



# Concept Practice Task - Negative

## Accessing Elements in a List of Integers

```
numbers = [5, 10, 15, 20, 25]

print(numbers[-5]) # Output: 5
print(numbers[-4]) # Output: 10
```

## Accessing Elements in a List of Booleans

```
bool_list = [True, False, True, False, True]

print(bool_list[-3]) # Output: True
print(bool_list[-5]) # Output: True
```

## Mixed Type

```
mixed_list = [42, "hello", 3.14, "world", True]

print(mixed_list[-1]) # Output: True
print(mixed_list[-3]) # Output: 3.14
```

## List Characteristics

- Order of Preservation
- Duplicates Are Allowed

# Order of Preservation

- When we create a list in Python, the **items are stored in the same order as we added them.**
- That order stays the **same unless we change it.**

## Creating a List

```
my_list = ["apple", "banana", "cherry"]  
  
print(my_list)  
# Output: ['apple', 'banana', 'cherry']
```

## Accessing Elements

```
my_list = ["apple", "banana", "cherry"]  
  
print(my_list[0]) # Output: apple  
print(my_list[1]) # Output: banana  
print(my_list[2]) # Output: cherry
```

### ✓ Key Points

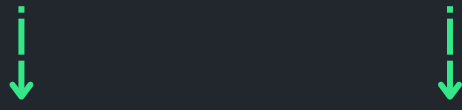
- Lists **remember the order of items** as they are added.
- Accessing elements by **index always returns them in the same order.**
- Even if you **update values**, the overall **order stays the same — unless you change it.**

# Duplicates Are Allowed

- Python lists can have the **same value appear more than once**.
- This is **useful when we want to store repeated data**.

## Creating a List with Duplicate Elements

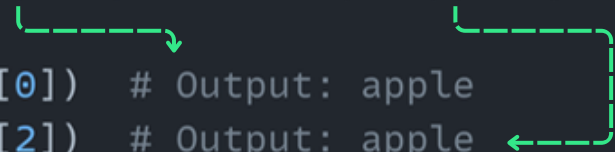
```
my_list = ["apple", "banana", "apple", "cherry", "banana"]
print(my_list)
```



# Output: ['apple', 'banana', 'apple', 'cherry', 'banana']

## Accessing Duplicate Values

```
my_list = ["apple", "banana", "apple", "cherry", "banana"]
print(my_list[0]) # Output: apple
print(my_list[2]) # Output: apple
print(my_list[1]) # Output: banana
print(my_list[4]) # Output: banana
```



Even though the **values are the same**, they are **treated as separate elements** based on their position.

### ✓ Key Points

- A list **can contain duplicates**
- Each **occurrence has its own index**
- **Duplicates** can be **accessed, counted, removed**

## Nested Lists

- What Are Nested Lists?
- Accessing Elements in Nested Lists
- Modifying Nested Lists



# What Are Nested Lists?

- A nested list is a list that **contains other lists as its elements**.
- This allows for a **multi-dimensional structure, similar to a table or matrix**.

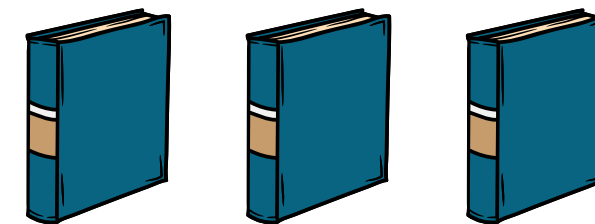
Let's see an example of Library



Library → outer list



Each bookshelf → inner list



Each Book → elements in the inner list

## Understanding - nested List

```
nested_list = [  
    [1, 2, 3], -----> First row  
    [4, 5, 6], -----> Second row  
    [7, 8, 9]  
]  
      |-----> Third row  
  
print(nested_list)
```

```
# Output: [ [1, 2, 3], [4, 5, 6], [7, 8, 9] ]
```

### Breakdown

- Outer list: []
- First inner list: [1, 2, 3]
- Second inner list: [4, 5, 6]
- Third inner list: [7, 8, 9]

- The **outer list contains three inner lists**, each of which **represents a row of data**.
- This is also **called a 2D list or matrix structure**.

# Concept Practice Task

GamutX

## Simple Nested List of Strings

```
nested_list = [
    ["apple", "banana", "cherry"],
    ["dog", "cat", "rabbit"],
    ["red", "green", "blue"]
]

print(nested_list)
# Output:
[['apple', 'banana', 'cherry'], ['dog', 'cat', 'rabbit'], ['red', 'green', 'blue']]
```

## Mixed Data Types

```
nested_list = [
    [1, "a", True],
    [2, "b", False],
    [3, "c", True]
]

print(nested_list)
# Output: [[1, 'a', True], [2, 'b', False], [3, 'c', True]]
```

## Nested List of Lists with Numbers and Strings

```
nested_list = [
    [10, 20, 30],
    ["one", "two", "three"],
    [100, 200, 300]
]

print(nested_list)
# Output: [[10, 20, 30], ['one', 'two', 'three'], [100, 200, 300]]
```

## Nested List with Floating Point Numbers

```
nested_list = [
    [1.1, 2.2, 3.3],
    [4.4, 5.5, 6.6],
    [7.7, 8.8, 9.9]
]

print(nested_list)
# Output: [[1.1, 2.2, 3.3], [4.4, 5.5, 6.6], [7.7, 8.8, 9.9]]
```

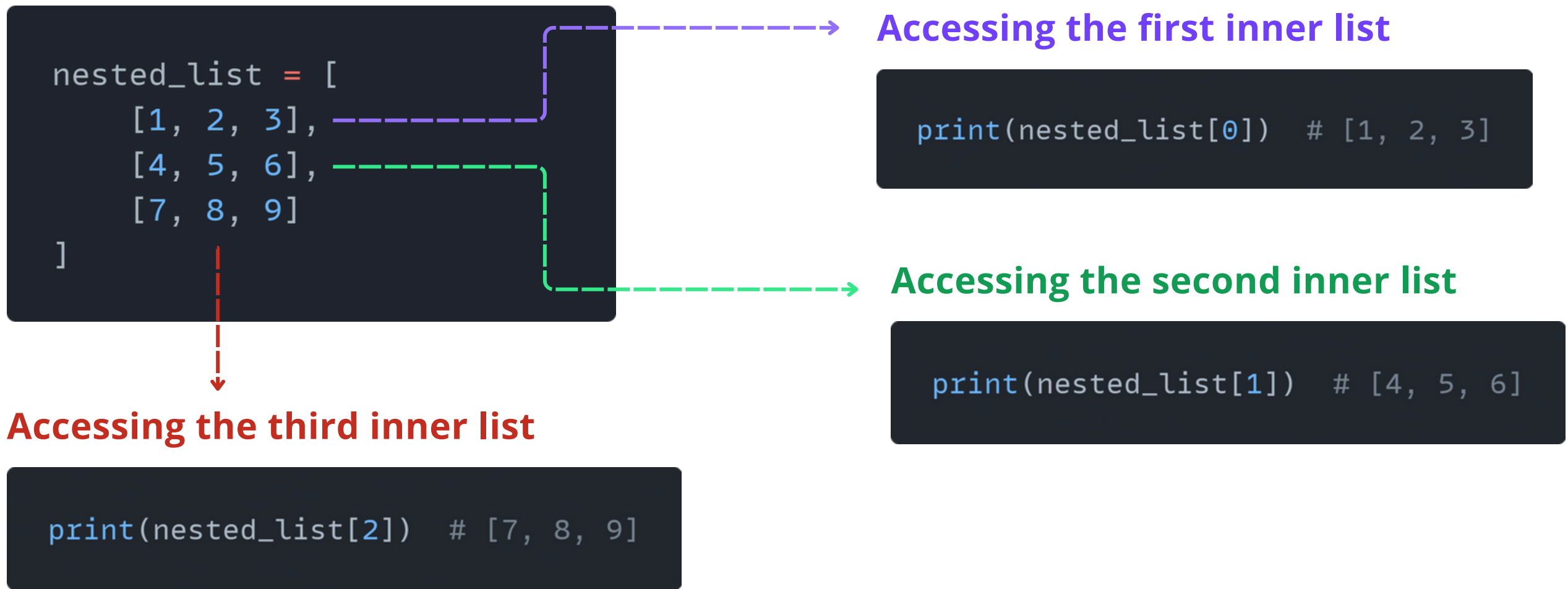


# Accessing Elements in Nested Lists

## Accessing an Entire Inner List

- To access an **inner list** from a nested list, we use the index of that inner list

```
nested_list = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```



Accessing the first inner list

```
print(nested_list[0]) # [1, 2, 3]
```

Accessing the second inner list

```
print(nested_list[1]) # [4, 5, 6]
```

Accessing the third inner list

```
print(nested_list[2]) # [7, 8, 9]
```

# Concept Practice Task

GamutX

## Nested List with Strings and a 2D List

```
mixed_list_2 = ["apple", "banana", ["x", "y"], ["cat", "dog"]]

print(mixed_list_2)
# Output: ['apple', 'banana', ['x', 'y'], ['cat', 'dog']]

print(mixed_list_2[0])    # Output: apple
print(mixed_list_2[1])    # Output: banana
print(mixed_list_2[2])    # Output: ['x', 'y']
```

## Mixed Data Types

```
mixed_list_1 = [10, "world", 3.14, [True, False]]

print(mixed_list_1)
# Output: [10, 'world', 3.14, [True, False]]

print(mixed_list_1[0])    # Output: 10
print(mixed_list_1[1])    # Output: world
print(mixed_list_1[2])    # Output: 3.14
print(mixed_list_1[3])    # Output: [True, False]
```

## Accessing Multiple Inner Lists

```
nested_list = [
    ["apple", "banana", "cherry"],
    ["dog", "cat", "rabbit"],
    [10, 20, 30]
]

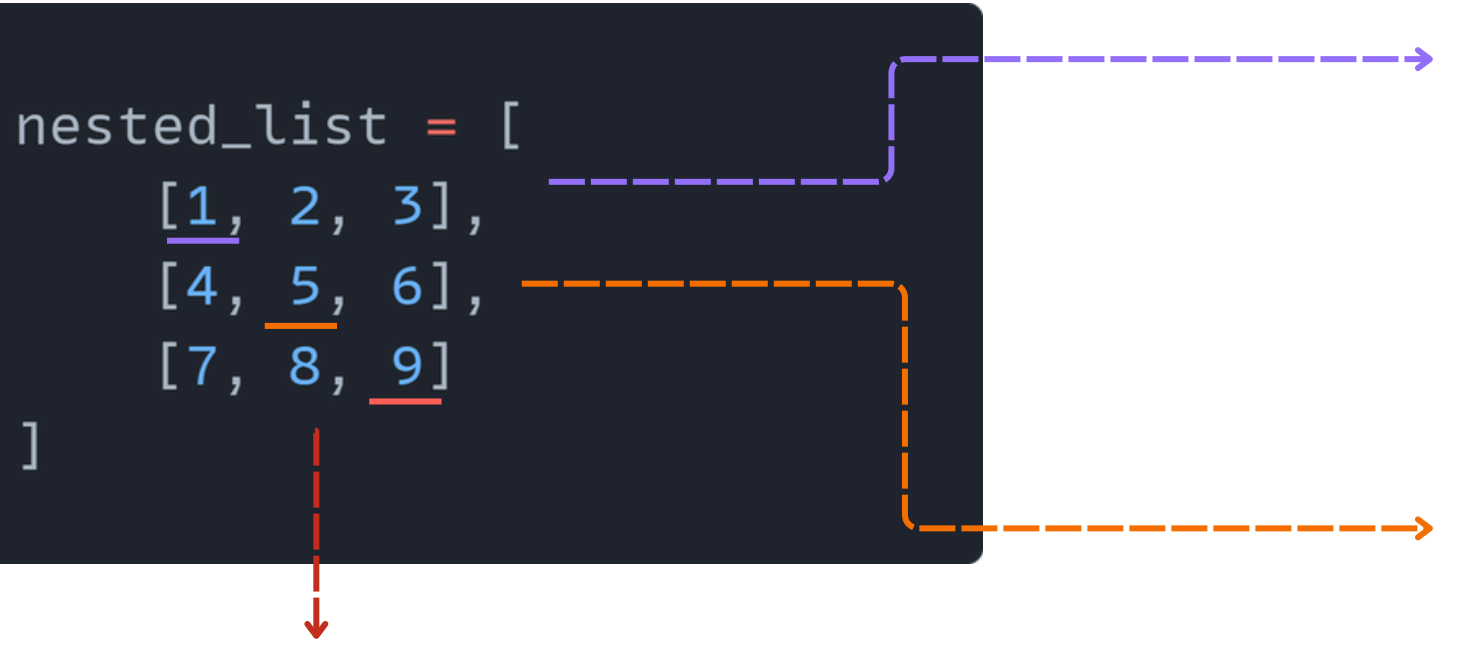
print(nested_list[0])
# Output: ['apple', 'banana', 'cherry']

print(nested_list[2])
# Output: [10, 20, 30]
```

## Accessing Elements in Nested Lists

- Nested lists **use two levels of indexing**:
  - **First index** → selects the inner list
  - **Second index** → selects the item inside that inner list

```
nested_list = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```



Access the third element of the third inner list

```
print(nested_list[2][2]) # Output: 9
```

Access the first element of the first inner list

```
print(nested_list[0][0]) # Output: 1
```

Access the second item in the second inner list:

```
print(nested_list[1][1]) # Output: 5
```

# Concept Practice Task

## 2D List with Mixed Data Types

```
two_d_list_2 = [  
    [1, "apple", 3.5],  
    ["banana", 2.5, 10],  
    [True, "grape", 7.5]  
]  
  
print(two_d_list_2[2]) # Output: [True, 'grape', 7.5]  
  
print(two_d_list_2[2][0]) # Output: True  
print(two_d_list_2[2][1]) # Output: grape  
print(two_d_list_2[2][2]) # Output: 7.5
```

## Simple 2D List with Integers

```
two_d_list_1 = [  
    [1, 2, 3], # First row  
    [4, 5, 6], # Second row  
    [7, 8, 9] # Third row  
]  
  
print(two_d_list_1[1]) # Output: [4, 5, 6]  
  
print(two_d_list_1[1][0]) # Output: 4  
print(two_d_list_1[1][1]) # Output: 5  
print(two_d_list_1[1][2]) # Output: 6
```

## List with Integers and Strings

```
nested_list_1 = [1, ["apple", "banana"], [3, 4, 5]]  
  
print(nested_list_1[1]) # Output: ['apple', 'banana']  
  
print(nested_list_1[1][0]) # Output: apple  
print(nested_list_1[1][1]) # Output: banana
```

# Modifying Nested Lists

- Modifying nested lists **involves changing the contents of an existing nested list.**
- This can be done by **accessing specific elements within the inner lists** and **assigning new values** to them.
  - **First index** → outer list (row)
  - **Second index** → inner list (column)

## Syntax

```
nested_list [ outer_index ][ inner_index ] = new_value
```

Use to access outer list

Use to access inner list



## Understanding - Modification

```
two_d_list = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print("Original List:")  
print(two_d_list)  
  
# Modify an element  
two_d_list[1][1] = 50  
  
print("\nModified List:")  
print(two_d_list)
```

### Original List:

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

### Modified List:

[[1, 2, 3], [4, 50, 6], [7, 8, 9]]

Final List

### 🔍 Explanation

- `two_d_list[1]` → accesses the second inner list: [4, 5, 6]
- `[1]` → accesses the second element in that list: 5
- `50` → replaces the value 5 with 50

# Concept Practice Task

GamutX

## Modifying Nested Lists with Integers and Strings

```
data_list = [
    [10, 20, 30],
    ["apple", "banana", "cherry"],
    [1.1, 2.2, 3.3]
]

print("Original List:")
print(data_list)
# Output: [[10, 20, 30], ['apple', 'banana', 'cherry'], [1.1, 2.2, 3.3]]

data_list[0][2] = 99

data_list[1][1] = "blueberry"

print("\nModified List:")
print(data_list)
# Output: [[10, 20, 99], ['apple', 'blueberry', 'cherry'], [1.1, 2.2, 3.3]]
```

## Nested List with Strings

```
string_list = [
    ["cat", "dog", "bird"],
    ["fish", "horse", "lion"]
]

print("Original List:")
print(string_list)
# Output: [['cat', 'dog', 'bird'], ['fish', 'horse', 'lion']]

string_list[1][2] = "tiger"

print("\nModified List:")
print(string_list)
# Output: [['cat', 'dog', 'bird'], ['fish', 'horse', 'tiger']]
```



# Concept Practice Task

GamutX

## Nested List with Mixed Data Types

```
mixed_list = [  
    [1, "hello"],  
    [2.5, "world"]  
]  
  
print("Original List:")  
print(mixed_list)  
# Output: [[1, 'hello'], [2.5, 'world']]  
  
mixed_list[1][0] = 10.5  
  
print("\nModified List:")  
print(mixed_list)  
# Output: [[1, 'hello'], [10.5, 'world']]
```

## Nested List with Booleans and Strings

```
bool_string_list = [  
    [True, False],  
    ["yes", "no"]  
]  
  
print("Original List:")  
print(bool_string_list)  
# Output: [[True, False], ['yes', 'no']]  
  
bool_string_list[0][1] = True  
  
print("\nModified List:")  
print(bool_string_list)  
# Output: [[True, True], ['yes', 'no']]
```



## List Operations

- Concatenating Lists
- Repeating Lists
- Using in Keyword
- Getting Length of a List

# Concatenating Lists

- Concatenation means joining two or more lists into a single list.
- We use the **+** operator to do this.

## Two Simple Lists

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

combined_list = list1 + list2
print(combined_list)
# Output: [1, 2, 3, 4, 5, 6]
```

## Lists with Different Data Types

```
numbers = [1, 2, 3]
words = ["apple", "banana", "cherry"]

combined = numbers + words
print(combined)
# Output: [1, 2, 3, 'apple', 'banana', 'cherry']
```

## Lists with Lists Inside

```
list1 = [1, [2, 3]]
list2 = [[4, 5], 6]

nested_combined = list1 + list2
print(nested_combined)
# Output: [1, [2, 3], [4, 5], 6]
```

## Multiple Lists

```
list_x = [7, 8, 9]
list_y = ['a', 'b', 'c']
list_z = [True, False]

all_combined = list_x + list_y + list_z
print(all_combined)
# Output: [7, 8, 9, 'a', 'b', 'c', True, False]
```

# Repeating Lists

- We can repeat the contents of a list multiple times using the `*` operator.

## List with Strings

```
# List to repeat
words = ["hello", "world"]

# Repeating the list
repeated_words = words * 3
print(repeated_words)

# Output:
# ['hello', 'world', 'hello', 'world', 'hello', 'world']
```

## Lists with Different Data Types

```
mixed_list = ['apple', 42, [1, 2]]

repeated_mixed = mixed_list * 2
print(repeated_mixed)

# Output: ['apple', 42, [1, 2], 'apple', 42, [1, 2]]
```

## Repeating an Empty List

```
empty_list = []

repeated_empty = empty_list * 5
print(repeated_empty) # Output: []
```

## List with Mixed Data Types

```
mixed_list = ['apple', 42, [1, 2]]

repeated_mixed = mixed_list * 2
print(repeated_mixed)

# Output: ['apple', 42, [1, 2], 'apple', 42, [1, 2]]
```

# Using in Keyword

- **Membership** testing checks if a value exists within a list. This is done using the **in** keyword.

## Membership test

```
my_list = [1, 2, 3, 4, 5]

is_in_list = 3 in my_list
print(is_in_list) # Output: True

is_not_in_list = 6 in my_list
print(is_not_in_list) # Output: False
```

## List with Nested Lists

```
nested_list = [[1, 2], [3, 4], [5, 6]]

is_nested_present = [3, 4] in nested_list
print(is_nested_present) # Output: True

is_nested_not_present = [7, 8] in nested_list
print(is_nested_not_present) # Output: False
```

## Mixed Data Types

```
mixed_data = [1, "hello", 3.14, True]

is_hello_present = "hello" in mixed_data
print(is_hello_present) # Output: True

is_2_71_present = 2.71 in mixed_data
print(is_2_71_present) # Output: False
```

## Checking with Strings

```
fruits = ["apple", "banana", "cherry"]

is_in_list = "banana" in fruits
print(is_in_list) # Output: True

is_not_in_list = "orange" in fruits
print(is_not_in_list) # Output: False
```

# Getting Length of a List

- If you want to know **how many elements are in a list**, you can **use the built-in len() function**.

## len() with List

```
my_list = [1, 2, 3, 4, 5]

list_length = len(my_list)
print(list_length) # Output: 5
```

## Basic List Length

```
numbers = [10, 20, 30, 40, 50]

length_of_list = len(numbers)
print(length_of_list)
# Output: 5
```

## Empty List

```
empty_list = []

length_of_empty = len(empty_list)
print(length_of_empty)
# Output: 0
```

## Mixed Data Types

```
mixed_list = [1, "hello", 3.14, True]

length_of_mixed = len(mixed_list)
print(length_of_mixed) # Output: 4
```

## List with Nested Lists

```
nested_list = [[1, 2], [3, 4], [5, 6]]

length_of_nested = len(nested_list)
print(length_of_nested) # Output: 3
```

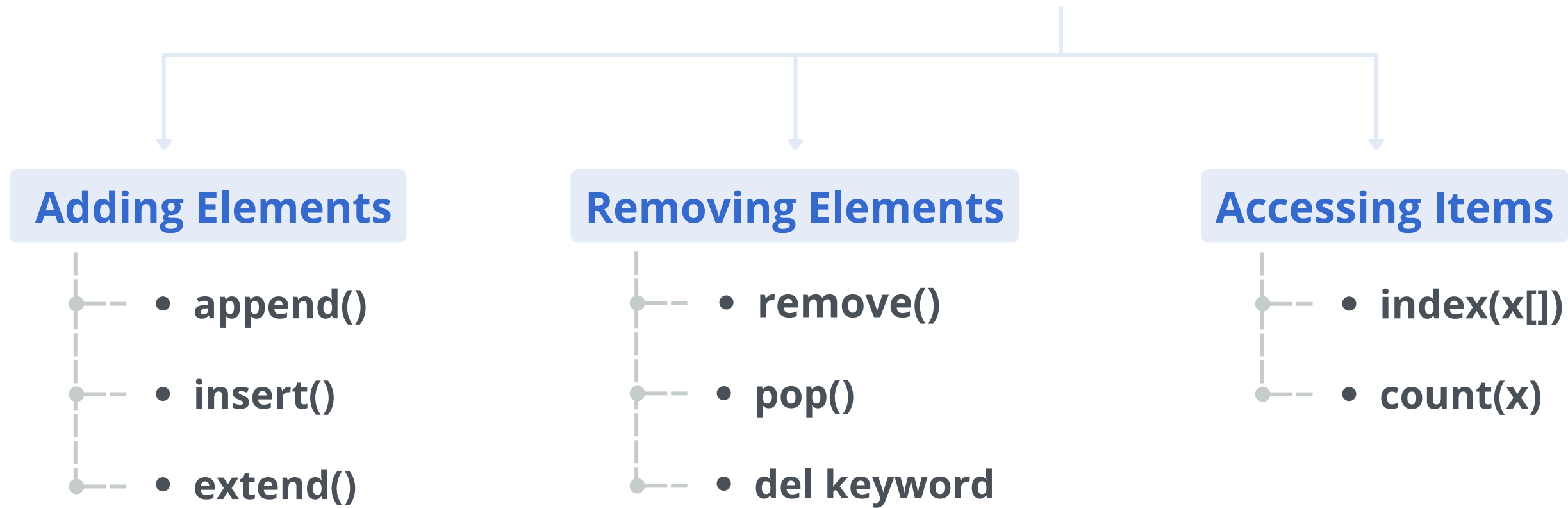


## List Methods

- `append()`
- `insert()`
- `extend()`
- `remove()`
- `pop()`
- `del` keyword
- `index(x[, start[, end]])`
- `count(x)`

# List Methods

- There are **8 types of list methods** we can categorize it in **3 category** as follow





# append()

- The append() method is **used to add a single element to the end of a list**.
- **append() always adds to the end of the list**
- **It adds one element at a time**

## Basic Usage

```
fruits = []  
  
fruits.append("apple")  
fruits.append("banana")  
  
print(fruits) # Output: ['apple', 'banana']
```

## Adding Different Data Types

```
mixed_list = []  
  
mixed_list.append(42)  
mixed_list.append("hello")  
mixed_list.append(3.14)  
  
print(mixed_list) # Output: [42, 'hello', 3.14]
```

## Appending a List as an Element

```
nested_list = []  
  
nested_list.append([1, 2, 3])  
  
print(nested_list) # Output: [[1, 2, 3]]
```

## Appending Elements to an Existing List

```
fruits = ["apple", "banana"]  
  
fruits.append("cherry")  
fruits.append("date")  
  
print(fruits)  
# Output: ['apple', 'banana', 'cherry', 'date']
```



# Concept Practice Task

## Adding a Single Item

```
numbers = []  
  
numbers.append(10)  
  
print(numbers) # Output: [10]
```

### Key Point:

- append() always adds to the end of the list
- It adds one element at a time

## Appending Nested Lists

```
nested_list = []  
  
nested_list.append([1, 2, 3])  
nested_list.append([4, 5, 6])  
  
print(nested_list)  
  
# Output: [[1, 2, 3], [4, 5, 6]]
```

# insert()

- The insert() method allows you to **add an element at a specific index in a list**.
- It **does not replace any element** — it just **shifts the existing elements to the right**.
- This method **takes two arguments**: the **index where you want to insert** the item and **the item itself**.

## Basic Syntax

```
list_name.insert(index, element)
```

item we want to add

Index where the item will be inserted

## Basic Usage

```
fruits = ["banana", "orange", "mango"]  
fruits.insert(0, "apple")  
print(fruits)  
# Output: ['apple', 'banana', 'orange', 'mango']
```

If the specified index is greater than the current length of the list, the element will be added at the end of the list.

# Concept Practice Task

GamutX

## Inserting in the Middle of a List

```
numbers = [1, 2, 4, 5]

numbers.insert(2, 3)

print(numbers) # Output: [1, 2, 3, 4, 5]
```

## Inserting Beyond the Length of the List

```
animals = ["dog", "cat"]

animals.insert(5, "fish")

print(animals) # Output: ['dog', 'cat', 'fish']
```

## Inserting Multiple Times

```
movies = ["Inception", "Avatar"]

movies.insert(1, "Titanic")

movies.insert(0, "The Matrix")

print(movies)

# Output: ['The Matrix', 'Inception', 'Titanic', 'Avatar']
```

## Inserting in an Empty List

```
empty_list = []

empty_list.insert(0, "Hello")

print(empty_list) # Output: ['Hello']
```

# extend()

- The extend() method is **used to add multiple elements to the end of a list**.
- It **takes another iterable (like a **list, tuple, or string**)** and **adds** each of its items to the **original list**.
- This method **modifies the original list** in place and does **not return a new list**.

## Basic Syntax

```
list_name.extend(iterable)
```

A collection of elements



## Basic Usage

```
colors = ["red", "blue"]  
  
colors.extend(["green", "yellow"])  
  
print(colors) # Output: ['red', 'blue', 'green', 'yellow']
```

## Extending with an Empty List

```
fruits = ["apple", "banana"]  
  
fruits.extend([])  
  
print(fruits) # Output: ['apple', 'banana']
```

**Note: It does not add the entire list as one item — it adds each element individually.**

# Concept Practice Task

## Extending with a List of Mixed Data Types

```
mixed_list = [1, "two", 3.0]
more_mixed = [False, "five", 6]

mixed_list.extend(more_mixed)
print(mixed_list)
# Output: [1, 'two', 3.0, False, 'five', 6]
```

## Extending with Another List

```
even_numbers = [2, 4, 6]

odd_numbers = [1, 3, 5]

even_numbers.extend(odd_numbers)

print(even_numbers) # Output: [2, 4, 6, 1, 3, 5]
```

## Extending with a String

```
chars = ['a', 'b']

chars.extend("cd")

print(chars)
# Output: ['a', 'b', 'c', 'd']
```



# remove()

- The remove() method is **used to delete the first occurrence of a specific value** from a list.
- If the **value is not found**, Python raises a **ValueError**
- **Only removes the first match**

## Basic Usage

```
fruits = ["apple", "banana", "cherry", "banana"]  
  
fruits.remove("banana")  
print(fruits) # ['apple', 'cherry', 'banana']
```

## Removing Elements from a List of Mixed Types

```
mixed_list = [1, "hello", 3.5, "world", 1]  
  
mixed_list.remove(1)  
  
print(mixed_list) # Output: ['hello', 3.5, 'world', 1]
```

## Remove a Non-Existent Element

```
vegetables = ["carrot", "broccoli", "spinach"]  
  
vegetables.remove("potato")  
  
print(vegetables)
```

```
# Output :  
# ERROR!  
# Traceback (most recent call last):  
#   File "<main.py>", line 5, in <module>  
# ValueError: list.remove(x): x not in list
```

# pop()

The pop() method is used to **remove an item from a list and return it**.

- If you **provide an index**, it **removes the item at that position**
- If you **don't provide an index**, it **removes the last item**

## Popping the Last Element

```
colors = ["red", "blue", "green"]
last_color = colors.pop()

print("Removed color:", last_color)
# Removed color: green

print("Updated list:", colors)
# Updated list: ['red', 'blue']
```

## Popping by Index

```
colors = ["red", "blue", "green"]
second_color = colors.pop(1)

print("Removed color:", second_color)
# Removed color: blue

print("Updated list:", colors)
# Updated list: ['red', 'green']
```

## Key Points

- pop() modifies the original list
- It also returns the item that was removed
- If the list is empty, using pop() will raise an IndexError



# Concept Practice Task

GamutX

## Popping from an Empty List

```
empty_list = []

popped_item = empty_list.pop()

print("Popped item:", popped_item)
```

```
# Output:
# ERROR!
# Traceback (most recent call last):
#   File "<main.py>", line 5, in <module>
# IndexError: pop from empty list
```

## Popping an Unknown Element

```
colors = ["red", "green", "blue"]

index_to_pop = 5
popped_color = colors.pop(index_to_pop)

print("Popped color:", popped_color)
```

```
# output :
# ERROR!
# Traceback (most recent call last):
#   File "<main.py>", line 8, in <module>
# IndexError: pop index out of range
```

## Popping an Element at a Specific Index

```
numbers = [10, 20, 30, 40]

second_number = numbers.pop(1)

print("Removed number:", second_number)
```

```
# Output: Removed number: 20
```

```
print("Updated list:", numbers)
```

```
# Output: Updated list: [10, 30, 40]
```



# del Keyword

The **del keyword** in Python is **used to delete**:

- A **specific item** in a list (by **index**)
- An **entire list**
- Or **even a variable**

Unlike **pop()**, it **does not return the deleted item** — it just removes it.

## Basic Syntax

```
del list_name[index]
```

- **del** → the keyword used to delete
- **list\_name** → the list you're working with
- **index** → the position of the item you want to remove

## Deleting an Element by Index

```
colors = ["red", "green", "blue", "yellow"]  
  
del colors[2]  
print(colors) # ['red', 'green', 'yellow']
```

# Concept Practice Task

## Deleting Elements from a Nested List

```
nested_list = [[1, 2, 3], ['a', 'b', 'c'], [True, False]]  
  
del nested_list[1]  
  
print(nested_list)  
  
# Output: [[1, 2, 3], [True, False]]
```

## Deleting an Element Using Negative Indexing

```
fruits = ["apple", "banana", "cherry", "date"]  
  
del fruits[-1]  
  
print(fruits) # Output: ['apple', 'banana', 'cherry']
```

## Deleting an Entire List

```
numbers = [1, 2, 3, 4, 5]  
  
del numbers  
  
print(numbers)  
print("The list 'numbers' has been deleted.")  
  
# output :  
# ERROR!  
# Traceback (most recent call last):  
#   File "<main.py>", line 7, in <module>  
# NameError: name 'numbers' is not defined
```

# index()

- The index() method is used to **find the position (index number) of the first occurrence of a specific value** in a list.
- If the **value is not found**, Python will raise a **ValueError**.

## Basic Usage

```
fruits = ["apple", "banana", "cherry", "date", "banana"]  
  
index_banana = fruits.index("banana")  
print(index_banana) # Output: 1
```

## Finding Index of a Nested List Element

```
nested_list = [["a", "b"], ["c", "d"], ["e", "f"]]  
  
index_inner = nested_list.index(["c", "d"])  
print(index_inner) # Output: 1
```

## Searching in a List of Mixed Data Types

```
mixed_data = [1, "apple", 3.14, "banana", 1]  
  
index_one = mixed_data.index(1)  
print(index_one) # Output: 0
```

## Find a Non-Existent Value

```
fruits = ["orange", "apple", "banana"]  
index_grape = fruits.index("grape")  
  
print(index_grape)  
  
# ERROR!  
# Traceback (most recent call last):  
#   File "<main.py>", line 5, in <module>  
# ValueError: 'grape' is not in list
```

# Concept Practice Task

GamutX

## Specifying Start and End Indices

```
grades = [90, 85, 90, 75, 85]

index_85 = grades.index(85, 3)
print(index_85)
```

# Output: 4

## Using All Parameters

```
numbers = [10, 20, 30, 40, 20, 50]

index_20 = numbers.index(20, 1, 4)
print(index_20)
```

# Output: 1

# count(x)

- The count() method is used to **count how many times a specific value appears in a list.**
- **if item is not found, it return zero ( 0 ) as output**

## Basic Syntax

```
fruits = ["apple", "banana", "apple", "banana", "apple"]  
  
apple_count = fruits.count("apple")  
print(apple_count) # Output: 3
```

## Counting a Non-existent Element

```
fruits = ["apple", "banana", "orange", "apple"]  
  
grape_count = fruits.count("grape")  
print(grape_count) # Output: 0
```

## Counting Numeric Elements

```
numbers = [1, 2, 2, 3, 4, 2, 5]  
  
count_of_twos = numbers.count(2)  
print(count_of_twos) # Output: 3
```

## Mixed Data Types

```
mixed_list = [1, "apple", 3.5, "apple", True, 1]  
  
count_of_ones = mixed_list.count(1)  
print(count_of_ones) # Output: 3
```

- **Here True also consider as 1**



# **Common Errors with Lists**

- Index Errors
- Type Errors
- Value Errors

# IndexError

- An IndexError occurs **when you try to access an index that doesn't exist in the list.**

## Causes

```
my_list = [10, 20, 30]

print(my_list[5])
```

```
⊗ PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/H
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    print(my_list[5])
IndexError: list index out of range
○ PS C:\Users\HP\Desktop\tt>
```

## 🔍 Why this happens:

- my\_list only has 3 elements → indexes: 0, 1, 2
- **Index 5 is out of range**

## ✅ How to Prevent It:

```
my_list = [10, 20, 30]

if len(my_list) > 5:

    print(my_list[5])
else:
    print("Index out of range.")
```

Make sure that the index you are trying to **access** is **within the valid range of the list.**



# TypeError

- A TypeError occurs when you try to perform an operation between incompatible data types.

## Causes

```
my_list = [10, 20, 30]

sum_value = my_list[0] + "hello"
```

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop/
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    sum_value = my_list[0] + "hello"
TypeError: unsupported operand type(s) for +: 'int' and 'str'
PS C:\Users\HP\Desktop\tt>
```

## ✓ How to Prevent It:

Ensure that the data types involved in operations are compatible.

```
sum_value = my_list[0] + 10
```

- **Change 'hello' to integer**

## 🔍 Why this happens:

- We cannot add a integer to String



# ValueError

- A ValueError occurs when you try to remove a value that doesn't exist in the list.

## Causes

```
my_list = [10, 20, 30]

my_list.remove(50)
print(my_list)
```

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    my_list.remove(50)
ValueError: list.remove(x): x not in list
PS C:\Users\HP\Desktop\tt>
```

## Why this happens:

- 50 is not in my\_list, so Python can't remove it

## ✓ How to Prevent It:

Use in to check before trying to remove:

```
my_list = [10, 20, 30]

if 50 in my_list:
    my_list.remove(50)
else:
    print("Value not found in the list.")

# Value not found in the list.
```

# Thank You

